

ONBRD-02: IAM and Authentication

Series: ONBRD — Dynatrace Onboarding | **Notebook:** 2 of 10 | **Created:** December 2025 | **Last Updated:** 09/24/2026

Setting Up Secure Access

Before inviting your team, configure authentication and permissions properly. This notebook covers SAML/SSO setup, API tokens, and the modern permission model.

Table of Contents

1. [Why IAM First?](#)
 2. [Authentication Options](#)
 3. [Configuring SAML SSO](#)
 4. [User Groups and Permissions](#)
 5. [API Token and OAuth Management](#)
 6. [Verification Queries](#)
 7. [Next Steps](#)
-

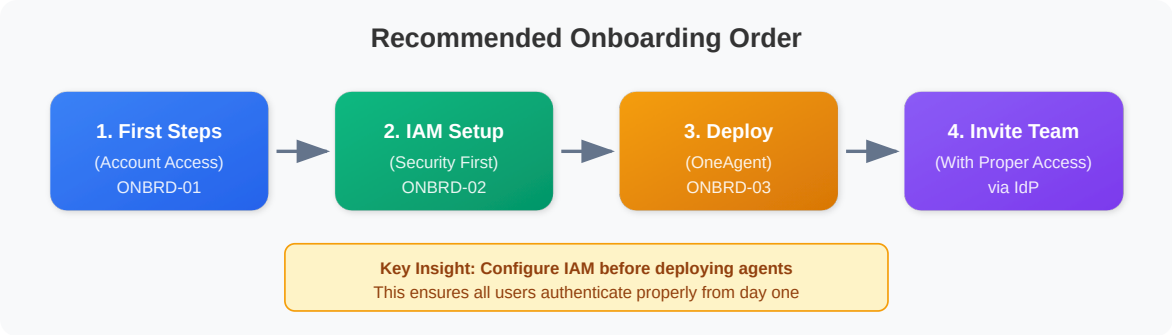
Prerequisites

- Account owner or admin access
- Identity provider details (for SSO)
- Understanding of your organization's access requirements

1. Why IAM First?

Setting up IAM before deploying OneAgent or inviting users ensures:

Benefit	Why It Matters
Consistent access	Users authenticate the same way from day one
Proper permissions	No accidental admin access for viewers
Audit trail	All access tied to corporate identity
Offboarding	Disabling IdP account revokes Dynatrace access
Compliance	Meet security requirements from the start

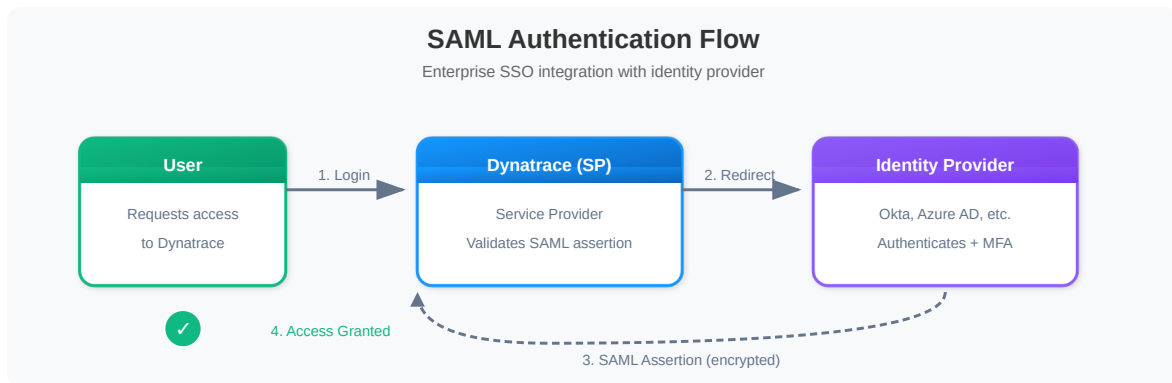


2. Authentication Options

Dynatrace supports multiple authentication methods:

Method	Description	Best For
Local Users	Built-in user accounts	Small teams, testing, break-glass accounts
SAML 2.0	Enterprise SSO integration	Most organizations
OIDC	OpenID Connect integration	Modern identity providers

Authentication Flow



Common Identity Providers

- **Azure Active Directory (Entra ID)** - Microsoft environments
- **Okta** - Cloud-native identity
- **OneLogin** - Enterprise identity
- **PingFederate** - On-premises/hybrid
- **Google Workspace** - Google-centric organizations

3. Configuring SAML SSO

Step 1: Access Account Management

Location: Account Management → Identity & access management → Single sign-on

You'll need these values for your IdP configuration:

Field	Description
Entity ID	Unique identifier for Dynatrace
ACS URL	Assertion Consumer Service URL
Relay State	Where users land after login

Step 2: Configure Your Identity Provider

In your IdP, create a new SAML application with:

1. **ACS URL** from Dynatrace
2. **Entity ID** from Dynatrace
3. **Name ID Format:** Email address (recommended)
4. **Required Attributes:**
 - `email` - User's email address
 - `firstName` - User's first name
 - `lastName` - User's last name

Step 3: Configure Dynatrace

Location: Account Management → Identity & access management → Single sign-on

1. Upload or paste the IdP metadata XML
2. Or manually configure:
 - **IdP Entity ID** - From your IdP
 - **SSO URL** - Login endpoint
 - **Certificate** - IdP signing certificate
3. Configure attribute mappings
4. Save and test

Step 4: Test SSO

Before enabling SSO for all users:

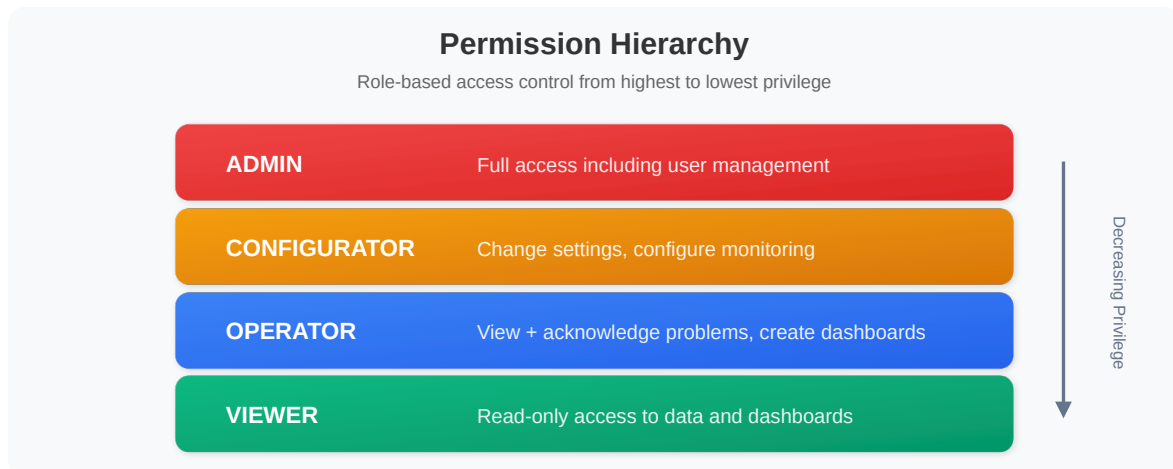
1. Open an incognito/private browser window
2. Navigate to your tenant URL (`https://{tenant-id}.apps.dynatrace.com`)
3. Select "Sign in with SSO"
4. Complete IdP authentication
5. Verify you land in Dynatrace with correct permissions

Warning: Keep at least one local admin account as a break-glass option in case SSO fails.

4. User Groups and Permissions

Permission Model

Dynatrace uses a policy-based access control model with the following permission levels:



Recommended Groups

Group	Role	Use Case
Platform Admins	Admin	Platform team, IAM management
SRE Team	Configurator	Workflow setup, monitoring config
Developers	Operator	Problem response, dashboards
Stakeholders	Viewer	Reports, read-only access

Creating Groups

Location: Account Management → Identity & access management → Groups

1. Click "Create group"
2. Name the group (e.g., "SRE-Team")
3. Add a description
4. Assign policies (permissions)
5. Optionally link to IdP groups for automatic membership

Scoping Access with Policies

The modern platform uses **policies** to control what users can access:

Policy Type	Purpose
Environment policies	Access to specific environments
Account policies	Account-level management
Data policies	Access to specific data (via segments + <code>dt.security_context</code>)

Parameterized Policies (Strongly Recommended)

Prefer **one parameterized policy bound to multiple groups via binding parameters** over many copies of the same policy with hardcoded scope values:

```
ALLOW storage:logs:read WHERE storage:dt.security_context =  
"${bindParam:team}";
```

Parameters are written as `${bindParam:<name>}` — Dynatrace's policy templating rule is *"Policy parameters should be prefixed with `bindParam:` and enclosed in `${...}`."* ([Policy templating \(DT docs\)](#)). Bind the policy to `team-payments` , `team-checkout` , `team-fraud` , etc., varying only the `team` parameter — one policy, N bindings. The parameter shape is load-bearing: design once, change rarely.

The `dt.security_context` field is the standardized boundary for Gen3 IAM scoping across both data and configurations. Without it, cross-entity-type policies cannot be written. Decide your `dt.security_context` value space before tagging anything (covered in **ONBRD-06**).

Where to Go Deeper

- **IAM-04 / IAM-05** — Designing effective policies and boundary conditions
- **IAM-10: Templated Policy-Group Assignments** — binding one templated policy to many groups with `bindParam` values
- **IAM-11 (WORKSHOP)** — Hands-on policy and persona design
- **IAM-99** — IAM best-practice summary and DQL reference
- **FAQ-02** — Tagging sources, standards, and strategy (`dt.security_context` design)
- **ORGNZ-06 / ORGNZ-07** — Security context for data and configuration scoping

Note: For data filtering, use **Segments + `dt.security_context`** (covered in ONBRD-06) rather than the legacy Management Zones.

5. API Token and OAuth Management

The modern platform supports three credential types — pick based on the integration:

Dynatrace API Credential Types

Choose by integration class — Sprint 1.337+ defaults

Platform Token	OAuth Client	Classic API Token
★ Recommended Default	External / Account-Admin	△ Legacy / Phase-Out
Prefix dt0s16 / dt0s01	Form Client ID + Secret	Prefix dt0c01
Auth Scheme Authorization: Bearer	Auth Scheme Authorization: Bearer	Auth Scheme Authorization: Api-Token
When to Use New automation Workflows + MCP OpenPipeline config Settings v2 Policy-based scoping → Default for net-new	When to Use External SaaS integrations Account-admin automation Above-tenant operations Short-lived bearer tokens Auto-rotation → External or admin	When to Use Existing scripts OneAgent installer (PaaS) Legacy integrations Scope-list permissions Migrate where possible → Legacy only

Wrong scheme returns 401 even with correct scopes/policies

Credential	Prefix / Form	When to Use
Platform Token <i>(recommended default for new automation)</i>	dt0s16	New automation, Workflows, MCP integrations, OpenPipeline configuration, Settings v2
OAuth 2.0 Client	client ID + secret	External SaaS integrations, account-admin automation
Classic API Token <i>(legacy phase-out)</i>	dt0c01	Existing scripts; migrate to Platform Token where possible

Platform Tokens (Recommended Default — Sprint 1.337+)

Platform Tokens are the standard for new automation as of sprint-337. They:

- Carry the **scopes you select** at creation, and only work within the permissions of the user (or service user) they are issued for — *"A platform token will only work within the limits of the assigned user's permissions."* ([Platform tokens \(DT docs\)](#))
- Are bound to a single user identity (or service identity) for traceability
- Should be the default unless you have a specific reason to choose OAuth or Classic

Location: My platform tokens ([myaccount.dynatrace.com/platformTokens](#)) for your own tokens; admins: **Account Management** → **Identity & access management** → **Platform tokens**

```
# Authorization scheme by token prefix:
#   dt0s16           → Authorization: Bearer <token> (dt0s01 is a
#                   separate SCIM account token, not a Platform Token)
#   dt0c01           → Authorization: Api-Token <token>;
# Wrong scheme returns 401 even with correct scopes/policies.
```

OAuth 2.0 Clients

Use for **external system integrations** or **account-admin automation** that operates above the tenant level.

Location: Account Management → OAuth clients

1. Create an OAuth client
2. Use client credentials flow to obtain bearer tokens
3. Tokens are short-lived and automatically rotated

Classic API Tokens (Legacy)

Existing scripts and OneAgent installer downloads still use Classic API tokens. Migrate to Platform Tokens during routine refresh cycles. Classic tokens use the `Api-Token` Authorization scheme (not `Bearer`).

Location: the **Access Tokens** app in your environment → **Generate new token** (not Account Management)

Common Use	Required Scope
OneAgent Installer Download	<code>InstallerDownload</code>
Metric Ingestion	<code>metrics.ingest</code>

Common Use	Required Scope
Log Ingestion	<code>logs.ingest</code>

Token Best Practices

Practice	Why
Minimal scope / least-privilege policy	Limit blast radius if compromised
Descriptive names	Know what each token is for
Expiration dates	Force rotation, reduce risk
Separate tokens per use	Revoke one without affecting others
Never commit to code	Use environment variables or secrets managers
Prefer Platform Tokens for new work	Aligned with the Gen3 IAM model

Where to Go Deeper

- **IAM series** — Token management, lifecycle, audit patterns

6. Verification Queries

After configuring IAM, verify your setup with these queries.

```
// Recent sign-ins to this environment (audit events)
// Data object corrected 09/24/2026. The Dynatrace audit trail is NOT in
`logs`: the former
// `fetch logs | filter matchesPhrase(log.source, "audit")` matched nothing,
or matched an
// unrelated file-based audit log (a database .aud file on the validation
tenant). Environment
// audit records are structured events in `dt.system.events` with event.kind
== "AUDIT_EVENT".
fetch dt.system.events, from:-24h
| filter event.kind == "AUDIT_EVENT"
| filter event.type == "LOGIN"
| fields timestamp, user.id, event.outcome, authentication.type, origin.type
| sort timestamp desc
| limit 50
```

```
// Sign-in outcomes over the last 7 days (audit events, not application logs)
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"LOGIN", "LOGOUT"})
| summarize events = count(), users = countDistinct(user.id), by:{event.type,
event.outcome}
| sort events desc
```

Manual Verification Checklist

Item	How to Verify
SSO working	Login via IdP in incognito window
Groups created	Check Account Management → Groups
Permissions assigned	Test with a viewer account
Break-glass account	Local admin login still works
API tokens	OneAgent deployment token ready

7. Next Steps

With IAM configured, you're ready to:

1. **ONBRD-03: Deploying ActiveGate** — Set up network routing (if needed for restricted networks)
2. **ONBRD-04: Cloud & SaaS Integrations** — Connect AWS / Azure / GCP and SaaS sources
3. **ONBRD-05: Deploying OneAgent** — Start collecting infrastructure and application data
4. **ONBRD-06: Organizing Your Environment** — Set up tags, segments, and `dt.security_context`
5. Invite team members via your IdP
6. Create additional Platform Tokens or OAuth clients as needed

IAM Tasks Before Moving On

- ☐ SAML/SSO configured and tested
- ☐ Break-glass local admin account documented
- ☐ User groups created for major roles
- ☐ Parameterized policy shape decided; group → policy bindings drafted
- ☐ `dt.security_context` value space planned (deeper in ONBRD-06 / FAQ-02)
- ☐ Platform Token issued for migration / automation tooling
- ☐ OneAgent installer (PaaS) token generated
- ☐ Token naming conventions established

Where to Go Deeper

- **IAM series** (15 notebooks) — full IAM administration depth
- **FAQ-02** — Tagging sources, standards, and strategy
- **ORGNZ series** — Bucket strategy, segments, security context

Summary

In this notebook, you learned:

- Why IAM should be configured before deploying agents
- Authentication options (Local, SAML, OIDC)

- How to configure SAML SSO
 - Permission levels, group structure, and parameterized policies
 - The three credential types (Platform Token, OAuth, Classic API Token) and when to use each
 - `dt.security_context` as the standardized boundary field for Gen3 IAM
 - How to verify IAM configuration
-

References

- [Identity and Access Management](#)
 - [Platform Tokens](#)
 - [Policy templating \(DT docs\)](#) — the `${bindParam:...}` syntax quoted in section 4
 - [SAML Configuration](#)
 - [Identity management — SSO and federation \(DT docs\)](#)
 - [Access Tokens](#)
 - [OAuth Clients](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.